

Lab 1: Python Environment Setup and Basic Assertions

This lab guides students through setting up their Python development environment, writing a simple Python function, and creating basic assertion scripts to programmatically verify its correctness. It introduces the fundamental concept of automated testing.

Lab Overview

Duration: 90 minutes

Type: guided

Language: Python

Learning Objectives

- • Set up Python development environment (VS Code)
- • Write a simple Python function (e.g., `add(a, b)`)
- • Write a separate small script to test it by calling with different inputs
- • Use `assert` statements to check expected outputs
- • Understand the idea of programmatic checking

Prerequisites

- • Basic computer literacy (navigating file systems, installing software)
- • Understanding of basic programming concepts: variables, functions, return values

Tools Required

Python (v3.8+)

Category: `ToolCategory.CLI_TOOL`

Installation: For Windows: Download from python.org and run installer (ensure 'Add Python to PATH' is checked).

For macOS: Download from python.org or use Homebrew (`brew install python3`).

For Linux: Usually pre-installed or install via package manager (`sudo apt-get install python3`).

Verification: `python3 --version`

Documentation: <https://www.python.org/downloads/>

Visual Studio Code (vLatest Stable)

Category: `ToolCategory.IDE`

Installation: Download from code.visualstudio.com and run installer.

Verification: `code --version` (after adding to PATH during install)

Documentation: <https://code.visualstudio.com/>

Python Extension for VS Code (vNone)

Category: `ToolCategory.OTHER`

Installation: Open VS Code, go to Extensions view (Ctrl+Shift+X), search for 'Python' by Microsoft, and click 'Install'.

Verification: Check for 'Python' extension in VS Code's Extensions view and ensure it's enabled.

Documentation: <https://marketplace.visualstudio.com/items?itemName=ms-python.python>

Setup Instructions

Step 1: Install Python

Install the latest stable version of Python 3.8 or newer on your operating system. Make sure to check the option to 'Add Python to PATH' during installation if you are on Windows.

Commands:

Windows: Download installer from python.org, run it, and check 'Add Python to PATH'.

macOS: Download from python.org or use ``brew install python3`` if Homebrew is installed.

Linux: ``sudo apt update && sudo apt install python3 python3-pip`` (for Debian/Ubuntu based systems).

Expected Output: A successful installation message or Python being available in your terminal.

⚠ Troubleshooting: If ``python3 --version`` doesn't work, ensure Python was added to your system's PATH. Re-run the installer and select the PATH option, or manually add it.

Step 2: Verify Python Installation

Open your terminal or command prompt and verify that Python is correctly installed.

Commands:

```
python3 --version
```

Expected Output: Python 3.x.x (where x.x is the version number, e.g., Python 3.9.7)

⚠ Troubleshooting: If you get 'command not found', refer to Step 1's troubleshooting or try ``python --version`` if ``python3`` isn't aliased.

Step 3: Install Visual Studio Code (VS Code)

Download and install VS Code, a popular code editor. It's recommended to add it to your system's PATH during installation.

Commands:

Download from: <https://code.visualstudio.com/>

Run the installer and follow the prompts. On Windows, ensure 'Add to PATH' is checked.

Expected Output: VS Code launching successfully after installation.

Step 4: Install Python Extension for VS Code

Open VS Code and install the official Python extension by Microsoft, which provides rich language support.

Commands:

1. Open VS Code.
2. Click on the Extensions icon in the Activity Bar on the side (or press ``Ctrl+Shift+X``).
3. Search for 'Python' and find the extension published by Microsoft.
4. Click 'Install'.

Expected Output: The Python extension showing as 'Installed' and 'Enabled' in the Extensions view.

⚠ Troubleshooting: If you can't find the extension, ensure you have an active internet connection. If it fails to install, try restarting VS Code.

Step 5: Create Your Lab Project Folder

Create a new folder on your computer to store all files for this lab. This helps keep your work organized.

Commands:

1. Open your file explorer/finder.
2. Navigate to a suitable location (e.g., your Documents folder).
3. Create a new folder named ``lab1_assertions``.
4. Open this folder in VS Code: ``File -> Open Folder...`` and select ``lab1_assertions``.

Expected Output: VS Code showing an empty `lab1\_assertions` folder in its Explorer view.

Lab Tasks

Task 1: Implement Basic Functions

🕒 Time: 15 min | 🎯 Difficulty: easy

You will create a Python file named `calculator.py` and implement two basic arithmetic functions: `add` and `subtract`.

Instructions:

1. In VS Code, right-click in the Explorer panel (on the `lab1_assertions` folder) and select 'New File'. Name it `calculator.py`.
2. Open `calculator.py` and add the following Python code to define the `add` function.
3. Implement the `subtract` function in `calculator.py` below the `add` function, following the same pattern. It should take two arguments and return their difference.

💡 Hints: • Remember that a function should `return` a value.
• The `add` function takes two parameters and returns their sum.

✅ Expected Outcome: A `calculator.py` file containing correctly implemented `add` and `subtract` functions.

Task 2: Create a Simple Test Script with Assertions

🕒 Time: 20 min | 🎯 Difficulty: easy

You will create a separate Python file (`test_calculator.py`) to test your `add` function using Python's built-in `assert` statement.

Instructions:

1. In VS Code, create another new file in your `lab1_assertions` folder and name it `test_calculator.py`.
2. In `test_calculator.py`, import the `add` function from `calculator.py`.
3. Write a simple test case for the `add` function: call `add(2, 3)` and use an `assert` statement to check if the result is `5`.
4. Add a `print` statement after the assertion to confirm the test passed (e.g., `print("Test 1 Passed!")`).
5. Run `test_calculator.py` from your terminal: `python3 test_calculator.py`.

💡 Hints: • The import statement should look like: `from calculator import add`.
• An `assert` statement has the form: `assert actual_value == expected_value`.

✅ Expected Outcome: The terminal should print 'Test 1 Passed!' if your `add` function is correct, or an `AssertionError` if it's wrong.

Task 3: Add More Assertions for `add` Function

🕒 Time: 20 min | 🌟 Difficulty: medium

Expand `test\_calculator.py` to include more assertion cases for your `add` function, covering different scenarios.

Instructions:

1. In `test\_calculator.py`, add at least three more `assert` statements for the `add` function:
 - Test with negative numbers (e.g., `add(-1, 5)` should be `4`).
 - Test with zero (e.g., `add(0, 0)` should be `0`).
 - Test with larger numbers (e.g., `add(100, 200)` should be `300`).
2. After each assertion, you can add a `print` statement indicating which test passed.
3. Run `test\_calculator.py` again (`python3 test\_calculator.py`) and ensure all tests pass.
4. (Optional challenge): Temporarily introduce a bug in your `add` function (e.g., `return a + b + 1`) and observe how the assertions catch it. Then fix the bug.

💡 Hints: • Think about edge cases: what if one number is negative? What if both are zero?
• The goal is to ensure the function works as expected across a range of valid inputs.

✅ Expected Outcome: All new assertions for the `add` function pass successfully without `AssertionError`.

Task 4: Test the `subtract` Function

🕒 Time: 20 min | 🌟 Difficulty: medium

Now, write assertions for your `subtract` function in `test\_calculator.py`.

Instructions:

1. In `test\_calculator.py`, import the `subtract` function: `from calculator import subtract`.
2. Add at least three `assert` statements to test the `subtract` function:
 - A basic positive subtraction (e.g., `subtract(5, 2)` should be `3`).
 - A subtraction resulting in a negative number (e.g., `subtract(2, 5)` should be `-3`).
 - Subtraction with zero (e.g., `subtract(10, 0)` should be `10`, `subtract(0, 5)` should be `-5`).
3. Run `test\_calculator.py` again (`python3 test\_calculator.py`) to confirm all tests for both `add` and `subtract` functions pass.

💡 Hints: • Ensure you import ``subtract`` alongside ``add`` at the top of your test file.

• Consider the order of arguments in subtraction: ``a - b`` is different from ``b - a``.

✅ Expected Outcome: All tests for both ``add`` and ``subtract`` functions pass successfully.

Checkpoints

Checkpoint 1: Python and VS Code Setup Verification

Success Criteria: Python 3.8+ is reported, and the Python extension is active in VS Code.

Verification Steps:

- Open a terminal and run ``python3 --version``.
- Open VS Code, navigate to Extensions, and confirm the Python extension is installed.

Checkpoint 2: Functions Implemented

Success Criteria: ``calculator.py`` contains both functions, and they return correct values when called directly.

Verification Steps:

- Check ``calculator.py`` for ``add`` and ``subtract`` function definitions.
- Run ``python3 -c 'from calculator import add, subtract; print(add(1,1)); print(subtract(2,1))'`` in terminal and check output.

Checkpoint 3: ``add`` Function Assertions Pass

Success Criteria: No ``AssertionError`` is raised for any of the ``add`` function tests, and all 'Test Passed!' messages for ``add`` are displayed.

Verification Steps:

- Run ``python3 test_calculator.py`` in the terminal.
- Observe the output for ``add`` function tests.

Checkpoint 4: All Assertions Pass

Success Criteria: No ``AssertionError`` is raised for any tests (both ``add`` and ``subtract`` functions), indicating all programmatic checks passed.

Verification Steps:

- Run ``python3 test_calculator.py`` in the terminal.
- Observe the final output.

Starter Code

 **calculator.py** Initial file for implementing calculator functions.

```
# Lab 1: Basic Calculator Functions

def add(a, b):
    """Adds two numbers and returns the sum."""
    # TODO: Implement the addition logic here
    pass

def subtract(a, b):
    """Subtracts the second number from the first and returns the difference."""
    # TODO: Implement the subtraction logic here
    pass
```

 **test_calculator.py** Initial file for writing assertion tests.

```
# Lab 1: Basic Assertion Script

# TODO: Import functions from calculator.py here

# --- Tests for add function ---
print("\n--- Testing add function ---")

# Test Case 1: Positive numbers
# TODO: Write an assertion for add(2, 3) == 5

# Test Case 2: Negative numbers
# TODO: Write an assertion for add(-1, 5) == 4

# Test Case 3: Zero
# TODO: Write an assertion for add(0, 0) == 0

# Test Case 4: Larger numbers
# TODO: Write an assertion for add(100, 200) == 300

print("All add function tests passed!")

# --- Tests for subtract function ---
print("\n--- Testing subtract function ---")

# Test Case 1: Basic subtraction
# TODO: Write an assertion for subtract(5, 2) == 3

# Test Case 2: Result is negative
# TODO: Write an assertion for subtract(2, 5) == -3

# Test Case 3: Subtracting zero
# TODO: Write an assertion for subtract(10, 0) == 10

# Test Case 4: Subtracting from zero
```

```
# TODO: Write an assertion for subtract(0, 5) == -5

print("All subtract function tests passed!")

print("\nAll tests completed successfully!")
```

Common Issues & Troubleshooting

****Python not found / 'python3' command not recognized****: Ensure Python is installed and added to your system's PATH. On Windows, you might need to re-run the installer and select the 'Add Python to PATH' option. On macOS/Linux, ensure your terminal is configured to use 'python3' or try 'python' instead.

****VS Code Python extension not working****: Make sure the extension is installed and enabled. Sometimes restarting VS Code helps. Ensure you've selected the correct Python interpreter in VS Code (bottom right corner of the status bar).

****`ModuleNotFoundError: No module named 'calculator'`****: This usually means you are not running the 'test_calculator.py' script from the same directory where 'calculator.py' is located. Navigate to your 'lab1_assertions' folder in the terminal before running 'python3 test_calculator.py'.

****`AssertionError`****: This means your function returned a value that did not match the expected value in your 'assert' statement. Carefully review your function's logic and the expected output.

Grading Rubric

This lab will be graded on the following criteria:

- ****Functionality (40%)****: 'add' and 'subtract' functions in 'calculator.py' work correctly.
- ****Assertions (40%)****: 'test_calculator.py' contains appropriate and sufficient 'assert' statements to cover various inputs for both functions.
- ****Code Quality (20%)****: Code is readable, well-formatted, and follows basic Python conventions (e.g., function names, variable names).

Submission Requirements

- The 'calculator.py' file containing the 'add' and 'subtract' functions.
- The 'test_calculator.py' file containing all assertion statements for both functions.

